Technical Requirements Guide

Scheduled Tasks UI for Ecommerce Application

1. Purpose

The system shall provide administrators with a user interface to create, manage, monitor, and audit scheduled tasks within the ecommerce platform.

The scheduled task feature will allow the application to automatically run backend jobs such as:

- order status sync
- abandoned cart reminders
- inventory synchronization
- payment reconciliation
- email or SMS notifications
- report generation
- product feed exports
- cleanup tasks
- retry failed integrations
- subscription or recurring business processes

This feature must support both one-time and recurring schedules.

---

2. Business Goals

The scheduled task module should:

- reduce manual administrative work
- improve consistency of recurring operations
- provide visibility into background automation
- support operational reliability and auditability
- allow controlled automation without requiring direct server access

---

3. High-Level Solution Overview

The solution should consist of 4 major parts:

A. React Admin UI

Used by administrators to:

- view all scheduled tasks
- create and edit task definitions
- enable or disable tasks
- run a task manually
- view execution history
- inspect failures and logs

B. Node.js API Layer

Used to:

- expose task configuration endpoints
- validate task definitions
- persist schedules and settings

- trigger manual executions
- return execution history and status

C. Scheduler/Worker Engine

Used to:

- poll or register due tasks
- execute task handlers
- manage retries
- record results
- prevent duplicate execution
- isolate long-running processes from the web server

D. Database Storage

Used to store:

- scheduled task definitions
- task parameters
- execution history
- status
- locks
- audit entries

---

4. Recommended Architecture

For a production ecommerce application, scheduling should not run directly inside the React frontend or only inside the main API request pipeline.

Recommended pattern:

- **React Admin App** = configuration and monitoring UI
- **Node.js API** = CRUD and orchestration endpoints
- **Worker Service** = dedicated background processor
- **Database** = persistent task storage
- **Optional Queue** = Redis-based queue for reliability and scale

Recommended flow

1. Admin creates a scheduled task in the React UI.
2. React sends the task definition to the Node API.
3. API validates and stores the schedule in the database.
4. Worker process checks for due tasks on a regular interval or via queue.
5. Worker executes the task handler.
6. Execution result is written to history tables.
7. UI displays current status, next run, last run, and logs.

---

5. Scheduling Model Options

Your system should support these schedule types:

5.1 One-Time Task

Run once at a specific date and time.

Examples:

- export orders tonight at 11:00 PM
- send a promotion on a specific date

5.2 Recurring Task

Run repeatedly on a defined pattern.

Examples:

- every 15 minutes
- every hour
- every day at 2:00 AM
- every Monday at 7:00 AM

5.3 Cron-Based Task

Advanced recurring schedule using cron expression.

Examples:

- `0 2 * * *` = every day at 2 AM
- `*/15 * * * *` = every 15 minutes

5.4 Event-Assisted Scheduled Task

A task may be scheduled but only runs when conditions are met.

Examples:

- retry failed payment sync every hour until success
- send follow-up email 2 days after order if delivery confirmed

---

6. Functional Requirements

6.1 Scheduled Task Management

The system shall allow administrators to:

- create a task
- edit a task
- delete a task
- enable or disable a task
- duplicate an existing task
- run a task immediately
- view all tasks
- filter tasks by type, status, enabled state, and frequency

Each task should include:

- task name
- task type
- description
- handler name or job key
- schedule type
- cron expression or date/time settings
- timezone
- enabled flag
- retry policy

- timeout setting
- concurrency setting
- task parameters
- last run date
- next run date
- last result
- created by
- updated by

6.2 Task Execution History

The system shall store execution history for each run.

History should include:

- task execution ID
- task ID
- started date/time
- completed date/time
- duration
- status
- message
- error details
- retry count

- triggered by
- worker instance
- correlation ID

Statuses should include at minimum:

- Pending
- Running
- Succeeded
- Failed
- Cancelled
- Skipped
- Retrying

6.3 Manual Execution

The system shall allow an authorized admin to manually run a task immediately.

Manual runs should:

- respect task parameters unless overridden
- record the user who initiated the run
- create a full execution history record
- optionally bypass next scheduled run logic if configured

6.4 Logging and Observability

The system shall provide logs for task execution.

Logs should include:

- task name
- execution timestamps
- major execution steps
- warning messages
- integration responses where safe
- exception stack traces for failures

The UI should support:

- viewing latest logs
- filtering by task and date
- searching log content
- downloading execution logs if needed

6.5 Failure Handling and Retries

The system shall support configurable retry rules.

Retry configuration should include:

- max retry attempts
- retry delay
- exponential backoff option
- dead-letter or failed state after max retries

The system should also support alerting for repeated failures.

6.6 Security and Authorization

Only authorized admin roles shall be allowed to:

- create or edit task definitions
- enable or disable tasks
- manually run tasks
- view sensitive logs
- delete tasks

Permissions should be role-based.

Example permissions:

- `ScheduledTasks.View`
- `ScheduledTasks.Create`
- `ScheduledTasks.Edit`
- `ScheduledTasks.Execute`
- `ScheduledTasks.Delete`
- `ScheduledTasks.Logs.View`

---

7. UI Requirements

7.1 Scheduled Tasks List Page

The UI should provide a grid showing:

- task name
- task type
- enabled status
- last run
- next run
- last result
- current state
- actions

Actions may include:

- Edit
- Enable/Disable
- Run Now
- View History
- View Logs
- Delete

Suggested filters:

- enabled/disabled
- succeeded/failed/running
- task category
- date range

- search by name

7.2 Task Create/Edit Form

The form should support:

- task name
- task type dropdown
- schedule type selector
- cron builder or cron expression input
- timezone selector
- start date
- optional end date
- JSON parameters or structured parameter fields
- retry policy settings
- timeout
- enabled checkbox

The form should include validation for:

- required fields
- valid cron syntax
- valid date ranges
- parameter schema validity
- duplicate task name rules if applicable

7.3 Execution History Page

Each task should have a history page showing:

- execution date/time
- status
- duration
- triggered by
- error summary
- retry count

Clicking a history row should show full execution details and logs.

7.4 Dashboard Widgets

Optional dashboard cards may show:

- total scheduled tasks
- active tasks
- failed runs in last 24 hours
- upcoming runs
- long-running tasks
- recent failures

---

8. Backend Technical Requirements

8.1 API Endpoints

Suggested REST endpoints:

Task Definitions

- `GET /api/scheduled-tasks`
- `GET /api/scheduled-tasks/:id`
- `POST /api/scheduled-tasks`
- `PUT /api/scheduled-tasks/:id`
- `DELETE /api/scheduled-tasks/:id`
- `PATCH /api/scheduled-tasks/:id/enable`
- `PATCH /api/scheduled-tasks/:id/disable`
- `POST /api/scheduled-tasks/:id/run`

History

- `GET /api/scheduled-tasks/:id/history`
- `GET /api/task-executions/:executionId`
- `GET /api/task-executions/:executionId/logs`

Metadata

- `GET /api/scheduled-task-types`
- `GET /api/timezones`

8.2 Task Handler Design

Each scheduled task should map to a backend handler.

Example handler keys:

- `inventory.sync`
- `orders.export`
- `notifications.abandoned-cart`
- `reports.daily-sales`
- `cleanup.temp-files`

Each handler should implement a consistent contract such as:

- validate input
- execute business logic
- log steps
- return structured result
- throw controlled exceptions on failure

8.3 Execution Isolation

Scheduled jobs should not depend on an active browser session.

They must run on the server side through a worker process.

Avoid putting business execution logic in the React UI.

8.4 Concurrency Control

The system shall prevent duplicate or overlapping runs when not allowed.

Options:

- lock by task ID
- allow single instance only
- allow limited parallel executions
- skip execution if prior run still active

This is especially important for:

- payment tasks
- stock updates
- report exports
- bulk email tasks

8.5 Idempotency

Where possible, scheduled handlers should be idempotent.

That means rerunning the same task should not create duplicate emails, duplicate exports, or repeated financial operations unless explicitly intended.

---

9. Data Model Requirements

9.1 ScheduledTasks Table

Suggested fields:

- `Id`
- `Name`
- `TaskKey`
- `Description`
- `Category`
- `ScheduleType`
- `CronExpression`
- `RunOnceAtUtc`
- `Timezone`
- `StartAtUtc`
- `EndAtUtc`
- `IsEnabled`
- `ParametersJson`
- `RetryMaxAttempts`
- `RetryDelaySeconds`
- `TimeoutSeconds`
- `AllowConcurrentRuns`
- `LastRunAtUtc`
- `NextRunAtUtc`

- `LastStatus`
- `CreatedBy`
- `CreatedAtUtc`
- `UpdatedBy`
- `UpdatedAtUtc`

9.2 TaskExecutions Table

Suggested fields:

- `Id`
- `ScheduledTaskId`
- `Status`
- `StartedAtUtc`
- `CompletedAtUtc`
- `DurationMs`
- `TriggeredByType`
- `TriggeredByUserId`
- `WorkerName`
- `AttemptNumber`
- `Message`
- `ErrorMessage`

- `ErrorStack`
- `CorrelationId`
- `CreatedAtUtc`

9.3 TaskExecutionLogs Table

Optional if you want step-by-step logs stored separately.

Suggested fields:

- `Id`
- `TaskExecutionId`
- `LogLevel`
- `Message`
- `Details`
- `CreatedAtUtc`

---

10. Scheduler Engine Options

You have a few common implementation patterns in Node.js.

Option A: Simple Polling Scheduler

A worker checks the database every few seconds for due tasks.

Best for:

- simple internal apps

- lower scale
- easy first implementation

Pros:

- straightforward
- database-driven
- easy to understand

Cons:

- more custom logic
- must handle locking carefully

Option B: Queue-Based Scheduler

Use Redis plus a queue library and push due jobs into workers.

Best for:

- higher reliability
- retries
- delayed jobs
- distributed workers

Pros:

- better scaling
- built-in retry patterns

- worker separation

Cons:

- more infrastructure
- Redis dependency

Option C: Hybrid

Store schedules in DB, compute due tasks in API/worker, dispatch execution through queue.

Best overall for an ecommerce app.

---

11. Recommended Implementation for Your App

For your ecommerce React + Node.js + API solution, I recommend this:

Phase 1

- React admin page for task management
- Node.js API for CRUD
- PostgreSQL or MSSQL tables for task storage
- dedicated Node worker service
- database-driven polling every 30–60 seconds
- execution history and logs

Phase 2

- add Redis queue

- add retries with backoff
- add alerts
- add dashboard widgets
- add cron expression builder
- add audit reporting

This gives you a simple first version without overengineering, while still leaving room to scale.

---

12. Non-Functional Requirements

The solution should meet these quality requirements.

Performance

- list page should load quickly with pagination
- history queries should be indexed
- task execution should not block normal API traffic

Reliability

- failed tasks must be recorded
- restarts should not lose task definitions
- due tasks should survive server restarts

Scalability

- architecture should support separate API and worker services
- support future multi-instance deployment

Security

- task parameters containing secrets must not be exposed in plain text
- only privileged roles may manage tasks
- logs must avoid exposing sensitive payment or credential data

Auditability

- all create, edit, delete, enable, disable, and manual run actions must be auditable

Maintainability

- task handlers should be modular
- new task types should be easy to register
- schedule calculation logic should be centralized

---

13. Validation Rules

The system shall validate:

- task name is required
- task key is required
- cron expression is valid when cron mode is selected
- one-time run date is required when one-time mode is selected
- timezone is required
- parameters are valid JSON if raw JSON is used
- retry settings are non-negative

- timeout is within allowed range
- duplicate active tasks are prevented where business rules require it

---

14. Audit Requirements

Audit events should be recorded for:

- task created
- task updated
- task deleted
- task enabled
- task disabled
- task run manually
- task execution failed
- retry exhausted

Audit record should include:

- user
- action
- entity
- old value summary
- new value summary
- timestamp

---

15. Example Ecommerce Scheduled Tasks

Examples you may want in your app:

Operations

- recalculate low-stock products every hour
- sync shipping statuses every 30 minutes
- clear expired carts nightly
- archive old logs weekly

Customer Communication

- send abandoned cart emails after 4 hours
- send review request emails 7 days after delivery
- send preorder reminder emails
- send SMS for high-value order shipment

Finance

- reconcile payment transactions nightly
- export daily sales summary
- detect failed payment captures

Catalog

- import inventory from suppliers
- refresh product feed
- disable expired promotions automatically

---

16. Technical Risks

Potential risks include:

- duplicate task runs in multi-server environments
- cron complexity causing misconfigured schedules
- long-running jobs impacting server resources
- poor retry handling causing repeated bad executions
- sensitive data leakage in logs
- timezone mistakes causing incorrect run times

Mitigation should include:

- distributed locking
- server-side validation
- separate worker process
- structured logging
- timezone-aware scheduling
- idempotent handlers

---

17. Suggested React UI Pages

You could structure the admin UI like this:

- `/admin/scheduled-tasks`

- `/admin/scheduled-tasks/new`
- `/admin/scheduled-tasks/:id/edit`
- `/admin/scheduled-tasks/:id/history`
- `/admin/task-executions/:executionId`

Reusable components:

- `ScheduledTaskTable`
- `ScheduledTaskForm`
- `CronExpressionBuilder`
- `TaskStatusBadge`
- `ExecutionHistoryTable`
- `ExecutionLogViewer`
- `RunTaskButton`
- `EnableDisableToggle`

---

18. Suggested Backend Modules

Example Node structure:

- `src/modules/scheduledTasks/controllers`
- `src/modules/scheduledTasks/services`
- `src/modules/scheduledTasks/repositories`

- `src/modules/scheduledTasks/models`
- `src/modules/scheduledTasks/dtos`
- `src/modules/scheduledTasks/validators`
- `src/modules/scheduledTasks/handlers`
- `src/workers`
- `src/jobs`
- `src/shared/logging`

Example handler organization:

- `handlers/inventorySyncHandler.ts`
- `handlers/abandonedCartHandler.ts`
- `handlers/dailySalesReportHandler.ts`

---

19. Acceptance Criteria

The feature will be considered complete when:

1. Admin can create a recurring or one-time task from the UI.
2. Task definitions are stored in the database.
3. Worker process executes due tasks automatically.
4. Admin can manually run a task.
5. System records execution history with success and failure states.

6. Admin can view next run, last run, and current status.
7. Failed jobs are logged with meaningful diagnostics.
8. Role-based permissions restrict task administration.
9. Tasks survive application restart.
10. Duplicate execution is prevented according to configuration.

---

20. Recommended Final Design Decision

For your stack, the cleanest approach is:

- **React admin UI** for configuration and monitoring
- **Node.js API** for CRUD and secure orchestration
- **separate worker service** for actual execution
- **database-backed schedules**